Advanced Data Structures and Algorithms

van Emde Boas Tree

Pedro Borges - up200207553
Gonçalo Pinto - up198704953
Manuel Mo - up202205000

Table of contents

01

Contextualization

Binary Trees

Definition: A hierarchical data structure where each node contains a value (like an integer) and connects at most **two children** (left and right).

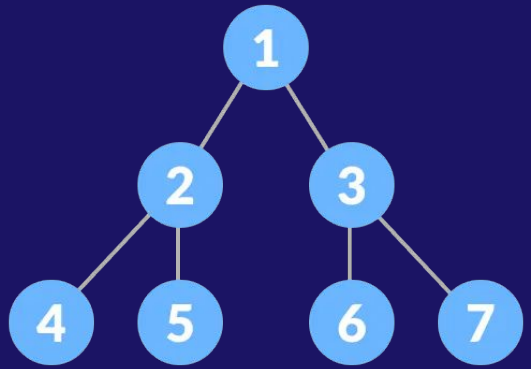
Search Property: Values smaller than the parent go to the **Left.** Values larger than the parent go to the **Right.**

Common use: We usually use them to keep a list of numbers sorted so we can find quickly the minimum, maximum or the successor.

In a balanced tree, finding a number takes log n steps.

Binary Trees

So, in a standard balanced Binary Search Tree, if we have **n** elements, it takes *log n* time to find the next number. This was the standard for decades. But what if it could be done quicker? What if we looked at the range of numbers instead of how many numbers we have? That's where the van Emde Boas tree comes in.



van Emde Boas Tree – Motivation I

A balanced binary tree has logarithmic complexity for **Insert, Delete, Search, Successor, and Predecessor** operations because its structure keeps the height proportional to **log(n)**. This prevents the tree from becoming skewed, so operations only traverse a root-to-leaf path in O(log n) time.

Operation	Complexity
Minimum	O(log n)
Successor	O(log n)
Insert	O(log n)
IsMember	O(log n)

van Emde Boas Tree – Motivation II

The van Emde Boas Tree (vEB for short) was invented in 1975 and it was created to break the O(log n) speed limit of traditional comparison-based priority queues (like binary heaps or balanced BSTs).

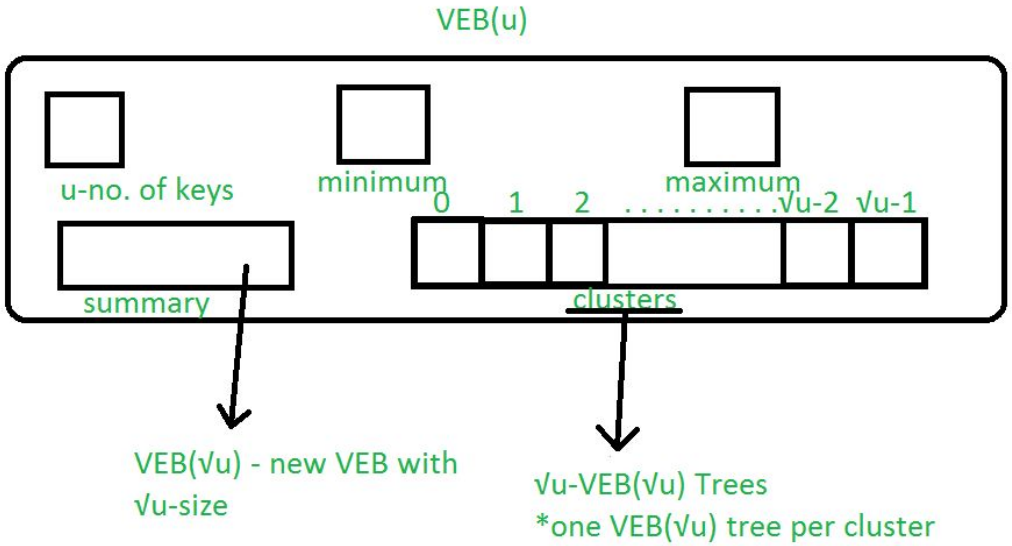
Note: Complexity for a universe of size $U = 2^k$

Operation	Complexity
Minimum	O(1)
Successor	O(log log U)
Insert	O(log log U)
IsMember	O(log log U)

02

Data Structure & Operations

Data Structure



Data Structure – Constructor

```
function vEB_Tree(u_power):
   this.universe_size_power = u_power
   this.min = this.max = -1

   if u_power == 1:  // Base case: universe size = 2
      this.summary = null
      this.clusters = null
   else:
      int clusters_num_power = ceil(u_power / 2)
      int cluster_size_power = floor(u_power / 2)

      this.summary = new vEB_Tree(clusters_num_power)
      this.clusters = new array of vEB_Tree*[2^clusters_num_power]

      for i = 0 to (2^clusters_num_power) - 1:
         this.clusters[i] = new vEB_Tree(cluster_size_power)
```

Data Structure – Helpers + Min/Max

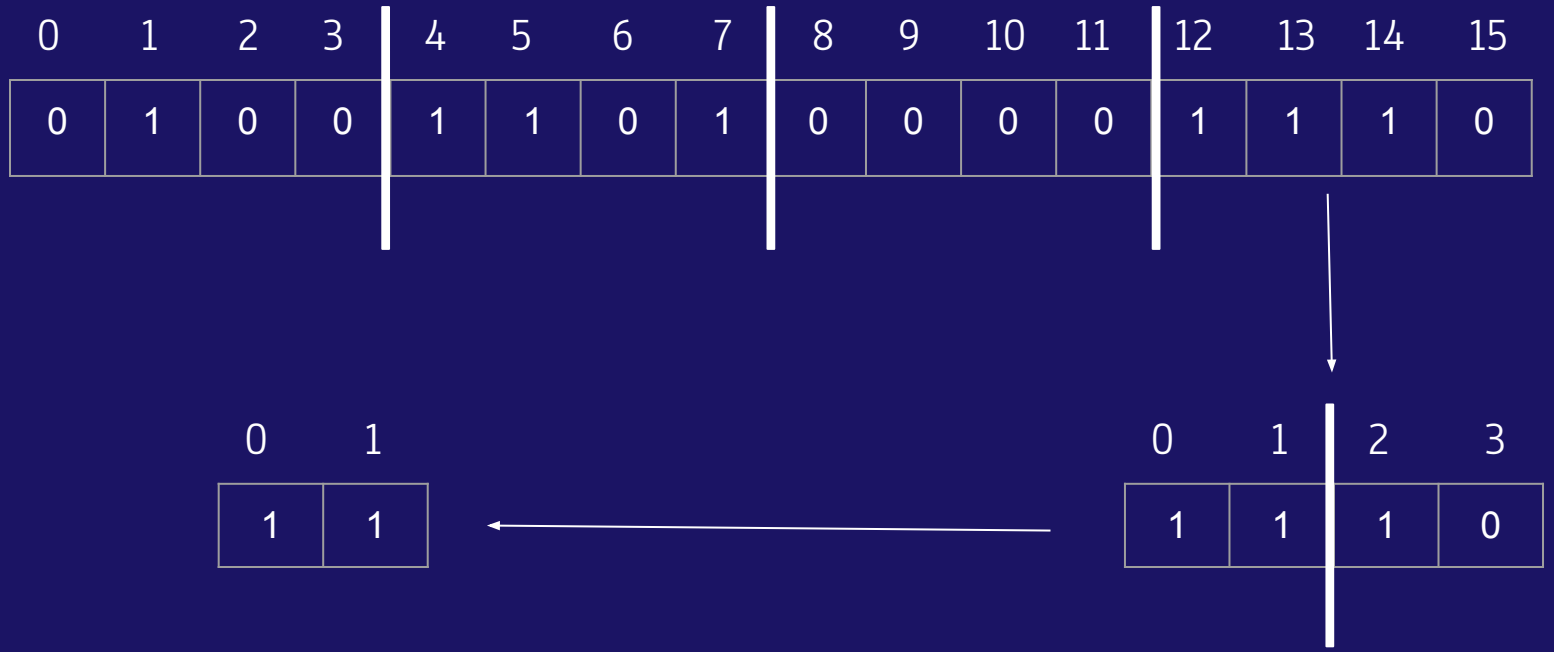
```
function high(x):
   // Returns the index of the cluster storing x
   return floor(x / 2^floor(this.universe_size_power/2))


function low(x):
   // Returns the value of x relative to its cluster
   return x mod 2^floor(this.universe_size_power/2)


function index(i, j):
   // Reconstructs value from cluster index and relative position
   return i * 2^floor(this.universe_size_power/2) + j


function getMin():
   return this.min


function getMax():
   return this.max
```

Data Structure – IsMember

```
function isMember(vEB, x):
   if x == vEB.min or x == vEB.max:
      return true

   if vEB.universe_size_power == 1:
      return false

   vEB_Tree targetCluster = vEB.clusters[vEB.high(x)]
   return isMember(targetCluster, targetCluster.low(x))
```

Data Structure – IsMember

isMember(13)

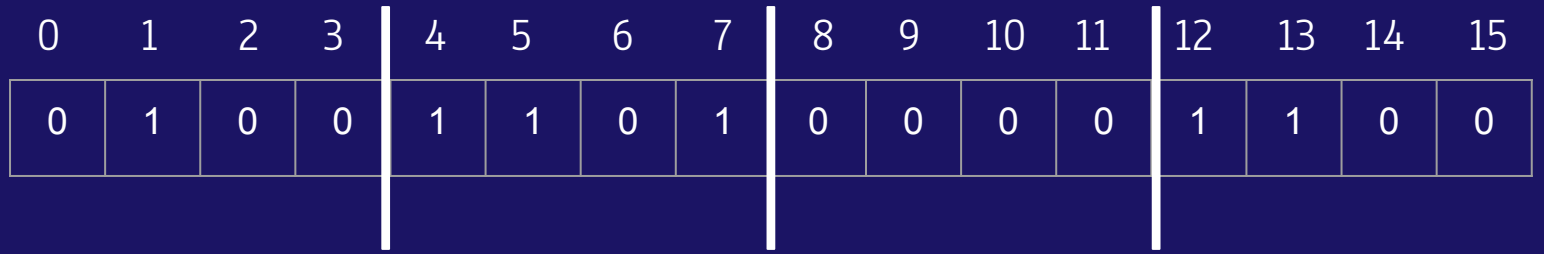


Data Structure – Successor

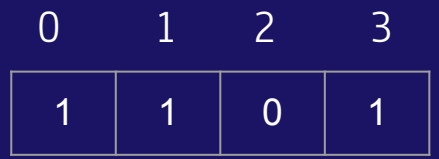
```
function successor(vEB, x):
   if x < vEB.min:
      return vEB.min

   if vEB.universe_size_power == 1:
      if x == 0 and vEB.max == 1:
         return 1
      else:
         return -1

   int targetCluster_index = vEB.high(x)
   int target
   vEB_Tree targetCluster = vEB.clusters[targetCluster_index]

   if targetCluster.low(x) < targetCluster.max:
      target = successor(targetCluster, targetCluster.low(x))
   else:
      targetCluster_index = successor(vEB.summary, targetCluster_index)

      if targetCluster_index == -1:
         return -1

      targetCluster = vEB.clusters[targetCluster_index]
      target = targetCluster.min

   return vEB.index(targetCluster_index, target)
```

Data Structure – Successor

Successor(7)



Summary:



Data Structure – Predecessor

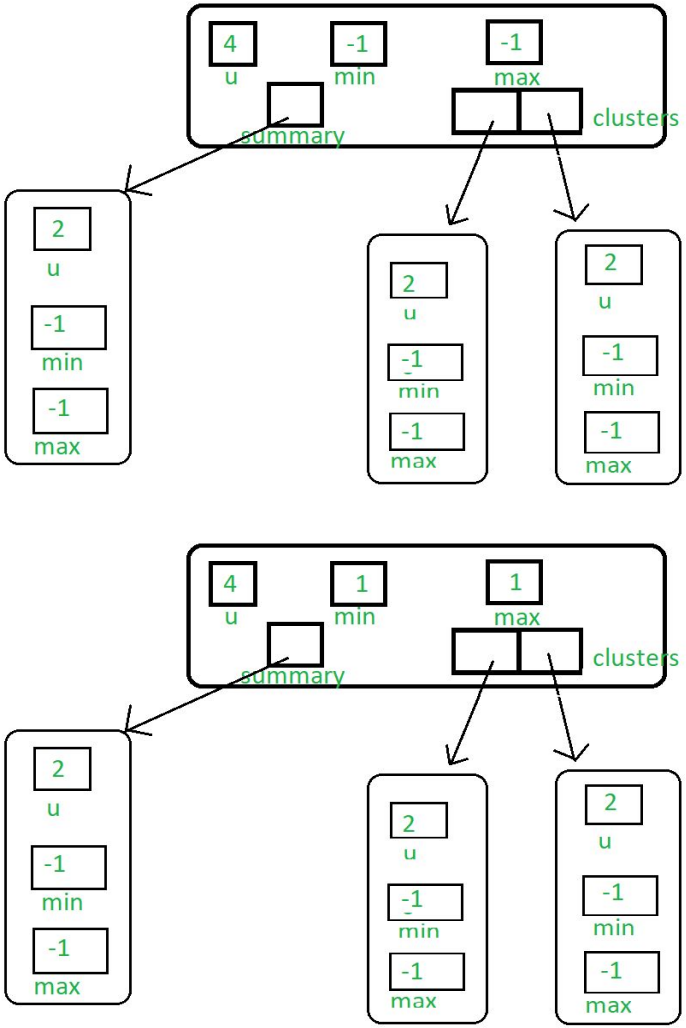
```
function predecessor(vEB, x):
   if x > vEB.max:
      return vEB.max

   if vEB.universe_size_power == 1:
      if x == 1 and vEB.min == 0:
         return 0
      else:
         return -1

   int targetCluster_index = vEB.high(x)
   int target
   vEB_Tree targetCluster = vEB.clusters[targetCluster_index]

   if targetCluster.low(x) > targetCluster.min:
      target = predecessor(targetCluster, targetCluster.low(x))
   else:
      targetCluster_index = predecessor(vEB.summary, targetCluster_index)

      if targetCluster_index == -1:
         if x > vEB.min:
            return vEB.min
         else:
            return -1

      targetCluster = vEB.clusters[targetCluster_index]
      target = targetCluster.max

   return vEB.index(targetCluster_index, target)
```

Data Structure – Insert

```
function insert(vEB, x):
   if vEB.min == -1:  // Empty tree
      vEB.min = x
      vEB.max = x
      return

   if x < vEB.min:  // Swap if inserting new minimum
      swap(x, vEB.min)

   if vEB.universe_size_power > 1:
      vEB_Tree targetCluster = vEB.clusters[vEB.high(x)]

      if targetCluster.min == -1:  // Cluster was empty
         insert(vEB.summary, vEB.high(x))

      insert(targetCluster, targetCluster.low(x))

   if x > vEB.max:
       vEB.max = x
```

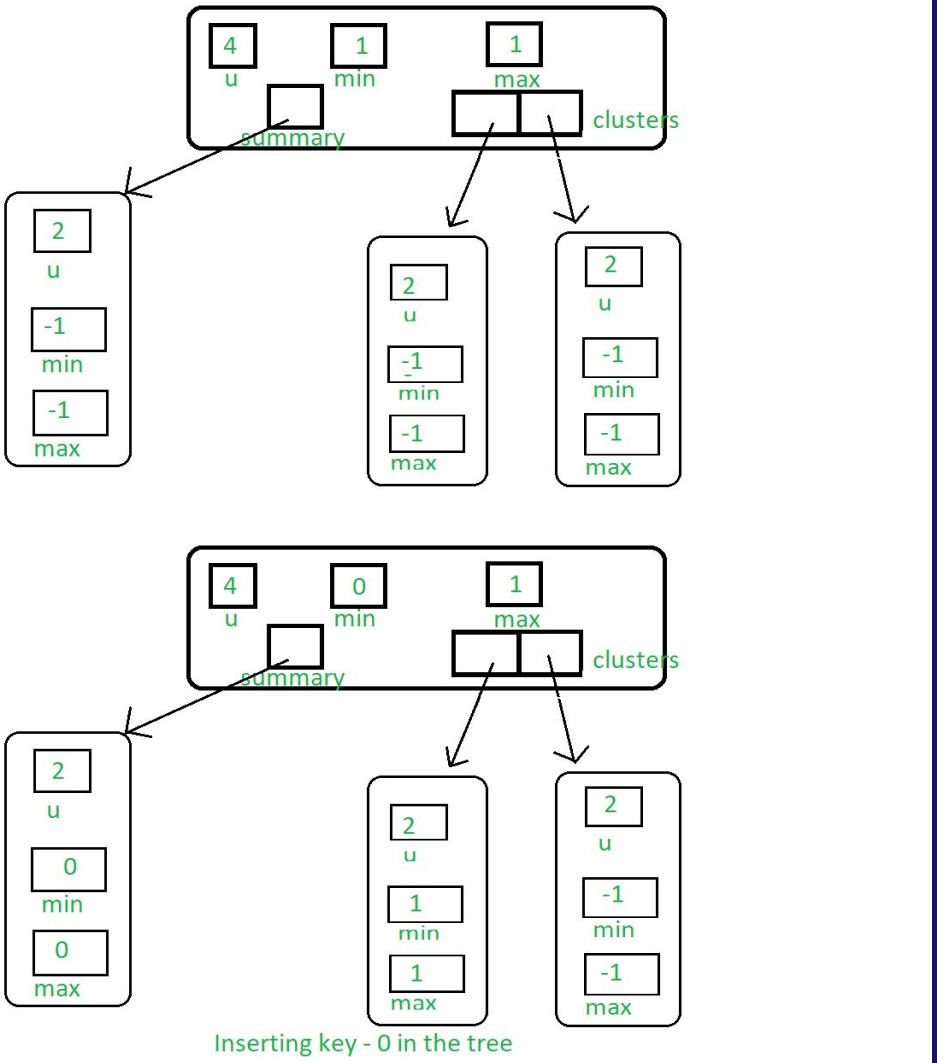
Data Structure – Insert

insert(1)



Data Structure – Insert

insert(0)



Data Structure – Delete

```
function delete(vEB, x):
   if vEB.max == vEB.min:  // Only one element
      vEB.min = -1
      vEB.max = -1
      return

   if vEB.universe_size_power == 1:  // Universe size = 2 with 2 elements
      vEB.min = (x == 0) ? 1 : 0
      vEB.max = vEB.min
      return

   if x == vEB.min:  // Special handling for minimum
      int min_cluster_index = vEB.summary.min
      x = vEB.clusters[min_cluster_index].min
      x = vEB.index(min_cluster_index, x)
      vEB.min = x

   vEB_Tree targetCluster = vEB.clusters[vEB.high(x)]
   delete(targetCluster, targetCluster.low(x))

   if targetCluster.min == -1:  // Cluster became empty
   delete(vEB.summary, vEB.high(x))

   if x == vEB.max:  // Update max if needed
      int max_cluster_index = vEB.summary.max
      if max_cluster_index == -1:
         vEB.max = vEB.min  // Only one element remains
      else:
         targetCluster = vEB.clusters[max_cluster_index]
         x = targetCluster.max
         x = vEB.index(max_cluster_index, x)
         vEB.max = x
```

03

Use Case Descriptions

Hash table comparison

Although both use the idea of mapping keys to positions and sacrifice space for speed, they have many differences:

Feature	vEB Tree	Hash Table
Lookup	O (log log U)	O(1) average O(n) worst case
Predecessor/successor	Fast	Not supported
Ordering	Sorted order	No order maintained
Key type	integers in fixed range	any hashable type
Summary	*“I know where everything belongs”*	*“I know exactly where I put it”*

Use Case Descriptions

van Emde Boas trees are primarily used in scenarios where you need extremely fast predecessor and successor queries on integer sets within a fixed range.

- Network routing;
- Graph Algorithms;
- Computational Geometry;
- Fast priority queues.

Use Case – Network routing

Routing table		
Range	->	Port
0.0.0.0	->	Port 4
10.0.0.0	->	Port 3
192.168.1.0	->	Port 1
192.168.2.0	->	Port 2

vEB Tree Stores: {0, 10.0.0.0, 192.168.1.0, 192.168.2.0}

PROBLEM IP Packet: 192.168.1.57 -> ?

SOLUTION predecessor(192.168.1.57) = 192.168.1.0

ANSWER 192.168.1.57 -> Port 1

PERFORMANCE

IPv4: $u = 2^{32}$ addresses

Lookup time: $O(\log \log u) \simeq 5$

04

Synthesis

Synthesis

WHEN TO USE

When you know you range, need the fastest possible lookup and can afford the space.

TRADE OFF

vEB trees use **O(u)** space, where u is the universe size (always 2^k), This can be much larger than **n** (the number of elements), **wasting significant memory** compared to a BST that uses **O(n)** space.

Limitations

- It requires a lot of space that makes it bad for large universes;
- Works only when it is in a fixed universe;
- The universe normally is a 2^k value, because the universe is divided in $\sqrt{u}$, high and low operations can be computed with bit shifts and each level halves the number of bits.
- Can be hard to implement because of its recursive design.

Conclusion

- Really good when faster operations are needed;
- Occupies more space than a BST;
- Hard to implement because of its recursive design;
- Works only in a fixed universe;
- The universe needs to be a 2^k value.

References

- Github with example code - https://github.com/Abdelaal1495/van-Emde-Boas-Tree
- Basic of vEB and it's construction - https://www.geeksforgeeks.org/dsa/van-emde-boas-tree-set-1-basics-and-construction/
- vEB operations - https://www.geeksforgeeks.org/dsa/van-emde-boas-tree-set-2-insertion-find-minimum-and-maximum-queries/
- Slides about vEB tree - https://web.stanford.edu/class/archive/cs/cs166/cs166.1166/lectures/14/Slides14.pdf

Q & A Time

Asymptotic analysis- IsMember

```
function isMember(vEB, x):                                  const:      time:
   if x == vEB.min or x == vEB.max:                         c1                1
      return true                                           c2                1

   if vEB.universe_size_power == 1:                         c3                1
      return false                                          c4                1

   vEB_Tree targetCluster = vEB.clusters[vEB.high(x)]       c5                1
   return isMember(targetCluster, targetCluster.low(x))     c6                { T(√u) + 1, n>2
                                                                                { 1, n<=2
```

Worst case:

$T(u) = c1+c2+c3+c4+c5+c6T(\sqrt{u}) + c6 \Leftrightarrow$

$T(u) = c6*\log(\log(u)) + c1+c2+c3+c4+c5+c6 = O(\log(\log(u)))$

T(u) $= c + T(u^{1/2}) = 2c + T(u^{1/4})$

After k steps: $T(u) = kc + T(u^{1/2^k})$

Base case: $u^{1/2^k} = 2 \Leftrightarrow \log(u^{1/2^k}) = \log(2) \Leftrightarrow \log(u)/2^k = 1 \Leftrightarrow \log(\log(u)) = k$

Asymptotic analysis- IsMember

Best case:
The function does not enter recursion. This occurs when *x* is either the minimum or maximum element, or when the initial cluster has a universe size of 2.
In this cases the function only evaluates conditional statements, resulting in a time complexity of O(1).

Average case:
Occasionally, x is the minimum or maximum early on, but most of the time the function still recurses. The probability that a randomly chosen element is equal to the minimum or maximum is extremely small. Therefore, the algorithm almost always follows the same execution path, reducing the universe from u to √u at each step, as in the worst case, leading to an average time complexity of O(log(log u)).

Asymptotic analysis- Successor

	const:	time:
function successor(vEB, x):		
if x < vEB.min:	c1	1
return vEB.min	c2	1
if vEB.universe_size_power == 1:	c3	1
if x == 0 and vEB.max == 1:	c4	1
return 1	c5	1
else:	c6	1
return -1	c7	1
int targetCluster_index = vEB.high(x)	c8	1
int target	c9	1
vEB_Tree targetCluster = vEB.clusters[targetCluster_index]	c10	1
if targetCluster.low(x) < targetCluster.max:	c11	1
target = successor(targetCluster, targetCluster.low(x))	c12	$\begin{cases} T(\sqrt{u}) + 1, & n>2 \\ 1, & n<= 2 \end{cases}$
else:	c13	1
targetCluster_index = successor(vEB.summary, targetCluster_index)	c14	$\begin{cases} T(\sqrt{u}) + 1, & n>2 \\ 1, & n<= 2 \end{cases}$
if targetCluster_index == -1:	c15	1
return -1	c16	1
targetCluster = vEB.clusters[targetCluster_index]	c17	1
target = targetCluster.min	c18	1
return vEB.index(targetCluster_index, target)	c19	1

Asymptotic analysis- Successor

Worst case:

$T(u) = c1+c2+c3+c4+c5+c6+c7+c8+c9+c10+c11+c12+c12T(\sqrt{u}) +c13+c14T(\sqrt{u})+c15+c16+c17+c18+c19 \Leftrightarrow$

$T(u) = (c12+c14)*\log(\log(u)) + c1+c2+c3+c4+c5+c6+c7+c8+c9+c10+c11+c12+c13+c14+c15+c16+c17+c18+c19 = O(\log(\log(u)))$

Base case: $u^{1/2^{\wedge}k} = 2 \Leftrightarrow \log(\log(u)) = k$

Best case:

The function does not enter recursion. This occurs when *x* is either lower than the minimum or when the initial cluster has a universe size of 2.

In this cases the function only evaluates conditional statements, resulting in a time complexity of O(1).

Average case:

The function recurses all the way to the bottom cluster unless it enters a cluster with a minimum value higher than x, making the average close to the worst case because most of the time the function will go to the base case where the cluster of size is 2 making the average case O(log(log(u))).

Asymptotic analysis- Insert

```
function insert(vEB, x):                                          const:          time:
   if vEB.min == -1:                                              c1                    1
      vEB.min = x                                                 c2                    1
      vEB.max = x                                                 c3                    1
      return                                                      c4                    1

   if x < vEB.min:                                                c5                    1
      swap(x, vEB.min)                                            c6                    1

   if vEB.universe_size_power > 1:                                c7                    1
      vEB_Tree targetCluster = vEB.clusters[vEB.high(x)]          c8                    1

      if targetCluster.min == -1:                                 c9                    1
         insert(vEB.summary, vEB.high(x))                         c10                   { T(√u) + 1, n>2
                                                                                          { 1, n<=2

      insert(targetCluster, targetCluster.low(x))                 c11                   { T(√u) + 1, n>2
                                                                                          { 1, n<=2

   if x > vEB.max:                                                c12                   1
       vEB.max = x                                                c13                   1
```

Asymptotic analysis- Insert

Worst case:

$T(u) = c1+c2+c3+c4+c5+c6+c7+c8+c9+c10+c10T(\sqrt{u})+c11+c11T(\sqrt{u})+c12 +c13 \Leftrightarrow$

$T(u) = (c10+c11)*\log(\log(u)) + c1+c2+c3+c4+c5+c6+c7+c8+c9+c10+c11+c12+c13 = O(\log(\log(u)))$

Best case:

The function does not enter recursion. This occurs when the vEB is empty. In this case the function only evaluates a conditional statement, resulting in a time complexity of O(1).

Average case:

vEB average insertion time is O(log log u) because the algorithm is restricted to exactly one recursive call per level. If the target cluster is empty, it recurses into the summary; if it isn't empty, it recurses into the cluster. By never doing both, the operation stays bound to the tree's height of log log u.